Compressed Sensing

Jacek Schmidt, jacek14schmidt@gmail.com

December 2025

1 Introduction

Modern scientific, industrial, and technological processes are fundamentally data driven. The acquisition, transmission, and storage of large volumes of information underpin applications ranging from digital imaging and communications to medical diagnostics and remote sensing. As data volumes continue to grow, reducing the amount of information that must be measured or transmitted without sacrificing reconstruction quality has become an increasingly important objective. Compressed sensing addresses this challenge by demonstrating that, under suitable structural assumptions, signals can be accurately reconstructed from far fewer measurements than traditionally thought necessary.

The central premise of compressed sensing is that many signals of practical interest are *sparse* or *compressible*: when represented in an appropriate basis, only a small number of coefficients carry significant information, while the remainder are zero or negligible. Although this requirement may appear restrictive, dense signals often correspond either to inherently incompressible data or to noise that is not essential for interpretation. Compressed sensing therefore exploits an underlying simplicity present in many real world datasets, providing a principled framework for reducing data dimensionality while preserving essential structure.

(a)

(b)

Figure 1: (a) Original image. (b) Reconstruction using only the largest 2% of the coefficients, with the remaining 98% set to zero.

Beyond storage considerations, data transfer itself is frequently the dominant contributor to energy consumption in modern systems, particularly when communication occurs over long distances. Transmitting fewer measurements can therefore yield substantial energy savings and simultaneously reduce the risk of data loss or corruption. Figure 1 illustrates this principle: a high–quality reconstruction is obtained from a dramatically reduced set of coefficients, demonstrating that aggressive compression need not imply significant loss of visual fidelity.

Compressed sensing is also relevant at the level of data acquisition. In classical sampling theory, such as the Shannon–Nyquist framework, a continuous signal is reconstructed from uniformly spaced discrete samples taken at a sufficiently high rate. In contrast, compressed sensing allows one to recover signals from fewer, nonadaptive measurements by exploiting sparsity in a suitable transform domain. This perspective has had a particularly strong impact in applications such as magnetic resonance imaging (MRI), computed tomography (CT), and spectroscopy, where physical or temporal constraints make dense sampling expensive or impractical. In these settings, compressed sensing enables faster acquisition while maintaining diagnostic quality. Figure 2 demonstrates the marked improvement achieved by compressed sensing reconstruction over conventional methods in the presence of undersampling.

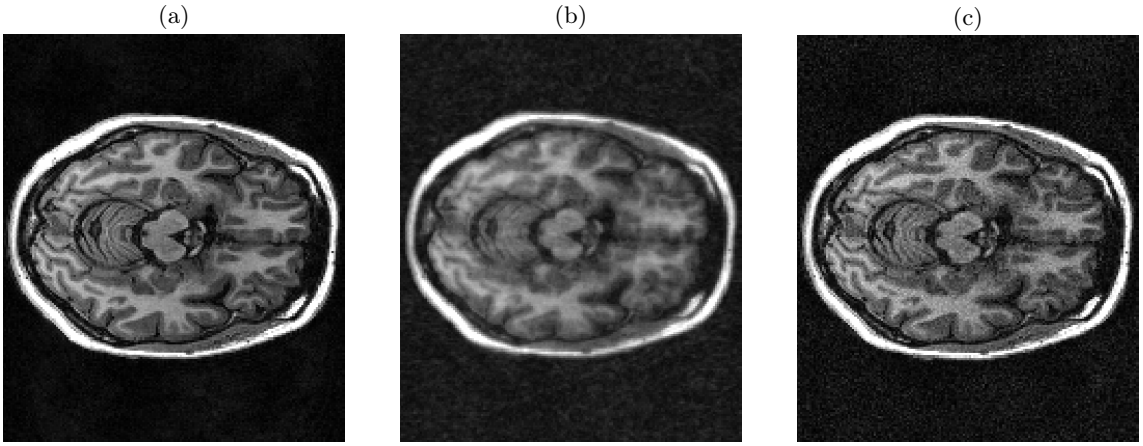

Figure 2: (a) Fully sampled image. (b) 4 × undersampled image with conventional reconstruction. (c) 4 × undersampled image with compressed snesing reconstruction.

From a mathematical standpoint, compressed sensing concerns the recovery of a vector $\mathbf{x} \in \mathbb{R}^n$ from a system of linear measurements

$$A\mathbf{x} = \mathbf{b},$$

where $A \in \mathbb{R}^{m \times n}$ with $m < n$. In data transmission, $\mathbf{b}$ represents a compressed encoding of the original signal $\mathbf{x}$; in sampling, it corresponds to a reduced set of measurements. Because the system is underdetermined, infinitely many vectors $\mathbf{x}$ are consistent with the measurements. Uniqueness is restored by restricting attention to sparse solutions, leading to a well–posed recovery problem under appropriate conditions on A and the sparsity level of $\mathbf{x}$. Although the recovered solution is, in general, only an approximation of the true signal, this approximation is often of remarkably high quality when the signal exhibits sufficient structure.

A wide range of algorithms has been developed to recover sparse solutions from underdetermined systems. Greedy methods, thresholding algorithms, convex ℓ_1–minimisation, and Bayesian approaches each offer different trade–offs between computational efficiency, theoretical guarantees, and robustness to noise and modelling error. Among these, thresholding–based methods strike a particularly attractive balance: they are computationally inexpensive, scale well to high dimensions, and are sufficiently simple to admit substantial modification and refinement.

This project focuses on iterative thresholding algorithms, with particular emphasis on Iterative Hard Thresholding (IHT) and its variants. We examine the theoretical conditions that guarantee the existence and uniqueness of sparse solutions, and analyse how algorithmic refinements such as random sensing matrices, column normalisation, gradient thresholding, and informed initialisation affect convergence behaviour and recovery performance. Through numerical experiments, we investigate the relationship between compression level, sparsity, and reconstruction accuracy, providing practical insight into how these algorithms may be deployed effectively in real–world applications.

2 Algorithm Formulation

2.1 Linear Systems

Compressed Sensing is concerned with recovering a vector $\mathbf{x}$ from a linear system

$$A\mathbf{x} = \mathbf{b}, \tag{1}$$

where $A \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. A standard numerical approach to solving such a system is to recast it as an optimisation problem. Rather than seeking an exact solution directly, one minimises the squared residual

$$\mathbf{r}^\top \mathbf{r} = (A\mathbf{x} - \mathbf{b})^\top (A\mathbf{x} - \mathbf{b}), \tag{2}$$

which provides a scalar measure of how well a given vector $\mathbf{x}$ satisfies the system. A minimiser is obtained when the gradient of the objective function vanishes. It is a routine calculation to show that the stationarity condition

$$\nabla_{\mathbf{x}} (\mathbf{r}^\top \mathbf{r}) = \mathbf{0}$$

is equivalent to the *normal equations*

$$A^\top A\mathbf{x} = A^\top \mathbf{b}. \tag{3}$$

Before discussing how iterative methods update their approximations, it is helpful to summarise the circumstances under which the normal equations admit a unique minimiser:

Overdetermined system ($m \geq n$) The normal equations have a unique solution precisely when $A^\top A$ is non-singular, which in this case is equivalent to A having full column rank.

Underdetermined system ($m \leq n$) The matrix $A^\top A$ is necessarily singular, so the normal equations admit infinitely many solutions. Each such solution yields the same minimal residual, although their Euclidean norms generally differ.

2.2 Steepest Descent

The classical method for numerically solving a linear system is the *steepest descent algorithm*. Although it is outperformed by more advanced methods in many settings, it has the appealing property that each iteration yields a guaranteed improvement, and does so in a locally optimal manner. This feature makes it a useful building block in the design of more specialised algorithms, such as Iterative Hard Thresholding (IHT), Normalised Iterative Hard Thresholding (NIHT), and the Conjugate Gradient Method (CGM). Its principal limitation is that it does not exploit structural features of the underlying system, and therefore fails to achieve globally efficient convergence.

To summarise the method, steepest descent updates the current approximation in the direction of greatest local decrease of the objective, determined by the gradient. Once the direction has been fixed, a line search is performed to determine an optimal step length along that direction. In the case of linear least-squares problems, these quantities admit closed-form expressions, which allows the algorithm to be implemented efficiently without explicitly recomputing the gradient or solving auxiliary optimisation problems at each step.

A typical implementation proceeds as follows. Starting from an initial guess $\mathbf{x}_0$, compute the initial descent direction

$$\mathbf{r}_0 = A^\top(\mathbf{b} - A\mathbf{x}_0).$$

Then iterate

$$\begin{aligned}
\alpha_i &= \frac{\mathbf{r}_i^\top \mathbf{r}_i}{\mathbf{r}_i^\top A^\top A\, \mathbf{r}_i}, \\
\mathbf{x}_{i+1} &= \mathbf{x}_i + \alpha_i \mathbf{r}_i, \\
\mathbf{r}_{i+1} &= A^\top(\mathbf{b} - A\mathbf{x}_{i+1}), \\
&= \mathbf{r}_i - \alpha_i A^\top A\, \mathbf{r}_i,
\end{aligned}$$

until the squared residual $\mathbf{r}_i^\top \mathbf{r}_i$ falls below a prescribed tolerance or a maximum number of iterations is reached.

In compressed sensing, however, we seek to recover a solution of an *underdetermined* linear system. Applying steepest descent directly would merely return one of infinitely many feasible solutions, as nothing in the framework enforces uniqueness or sparsity. Consequently, the method cannot be used on its own to recover the specific solution of interest.

2.3 Sparsity

The obstacle of non-uniqueness in underdetermined linear systems can be overcome by restricting attention to *sparse* solutions. Imposing a sparsity constraint reduces the admissible solution set, creating the possibility of recovering a unique minimiser.

Definition (k-sparse) A vector is said to be k-sparse if it contains at most k non-zero entries.

A simple but useful observation is that if a vector $\mathbf{x}$ is k-sparse, then it is also l-sparse for any $l \geq k$. We now introduce a fundamental uniqueness result for sparse recovery.

Theorem 1. *Let $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$ and $A \in \mathbb{R}^{m \times n}$ with $m < n$. If $\operatorname{spark}(A) = m + 1$ and $2k \leq m$, then the minimisation problem*

$$\mathbf{r}^\top \mathbf{r} = (A\mathbf{x} - \mathbf{b})^\top (A\mathbf{x} - \mathbf{b})$$

admits a unique k-sparse minimiser.

Proof. Assume for contradiction that $\mathbf{x}_1$ and $\mathbf{x}_2$ are distinct k-sparse minimisers. Both satisfy the normal equations:

$$A^\top A\mathbf{x}_1 = A^\top \mathbf{b}, \qquad A^\top A\mathbf{x}_2 = A^\top \mathbf{b}.$$

Subtracting yields

$$A^\top A(\mathbf{x}_1 - \mathbf{x}_2) = \mathbf{0}.$$

Let $S = \operatorname{supp}(\mathbf{x}_1 - \mathbf{x}_2)$. Since each vector is k-sparse,

$$\|S\| \leq \|\operatorname{supp}(\mathbf{x}_1)\| + \|\operatorname{supp}(\mathbf{x}_2)\| \leq 2k \leq m.$$

Restricting to the support S gives

$$[A^\top A]_S(\mathbf{x}_1 - \mathbf{x}_2) = \mathbf{0},$$

where $[A^\top A]_S$ denotes the submatrix formed from the columns indexed by S.

The columns of A are linearly independent if and only if the corresponding columns of $A^\top A$ are linearly independent. Since $\operatorname{spark}(A) = m + 1$, every collection of at most m columns of A is linearly independent, and hence the same holds for $A^\top A$. As $\|S\| \leq m$, it follows that the only vector in the nullspace of $[A^\top A]_S$ is the zero vector. Thus $\mathbf{x}_1 = \mathbf{x}_2$, a contradiction. □

This theorem shows that, provided A satisfies a mild structural condition (satisfied by effectively all randomly generated matrices) and the selected sparsity level is not too large, a unique sparse solution can always be identified.

It should be noted that sparse recovery is only meaningful when the original vector is itself sparse—or well approximated by a sparse vector whose sparsity is comparable to the sparsity used in the solution space. Otherwise, the sparse solution will be a poor representation of the true signal.

Having established conditions for uniqueness, we now turn to computation. Steepest descent alone cannot enforce sparsity and would converge to an arbitrary solution of the underdetermined system. A naïve alternative is to guess the support of the true vector and apply steepest descent to that restricted system, but this requires checking all $\binom{n}{k}$ possible supports, which is computationally infeasible. A more practical approach is to update iterates in a way that preserves sparsity while reducing the residual, without committing to a fixed support.

2.4 Iterative Hard Thresholding (IHT)

Iterative Hard Thresholding (IHT) combines the classical steepest descent method with an explicit enforcement of sparsity. It forms the basis of many more advanced algorithms used for sparse recovery in underdetermined linear systems.

Each iteration consists of two conceptually separate steps. First, a steepest descent step is taken in order to reduce the squared residual as much as possible along the current gradient direction. Second, the resulting intermediate vector is projected onto the set of k-sparse vectors, ensuring that the sparsity constraint is satisfied at every full iteration. This projection is performed by the *hard thresholding* operator, denoted $\mathcal{H}_k$.

Let $\mathcal{S}_k$ denote the set of k-sparse vectors in $\mathbb{R}^n$. The operator $\mathcal{H}_k$ is defined by

$$\mathcal{H}_k : \mathbb{R}^n \to \mathcal{S}_k, \qquad \mathbf{v} \mapsto \mathbf{u} \quad \text{such that}$$

$$\|\mathbf{v} - \mathbf{u}\| \leq \|\mathbf{v} - \mathbf{w}\| \quad \forall\, \mathbf{w} \in \mathcal{S}_k.$$

Equivalently, $\mathcal{H}_k(\mathbf{v})$ retains the k largest entries of $\mathbf{v}$ in magnitude and sets all others to zero.

Starting from an initial guess $\mathbf{x}_0$, define the initial gradient

$$\mathbf{r}_0 = A^\top(\mathbf{b} - A\mathbf{x}_0).$$

For each iteration i, compute

$$\alpha_i = \frac{\mathbf{r}_i^\top \mathbf{r}_i}{\mathbf{r}_i^\top A^\top A\, \mathbf{r}_i},$$

$$\mathbf{x}_{i+1} = \mathcal{H}_k(\mathbf{x}_i + \alpha_i \mathbf{r}_i)\,,$$

$$\mathbf{r}_{i+1} = A^\top(\mathbf{b} - A\mathbf{x}_{i+1}),$$

and continue until the squared residual $\mathbf{r}_i^\top \mathbf{r}_i$ falls below a prescribed tolerance or a maximum number of iterations is reached. Figure 3 illustrates the progression of the IHT algorithm.

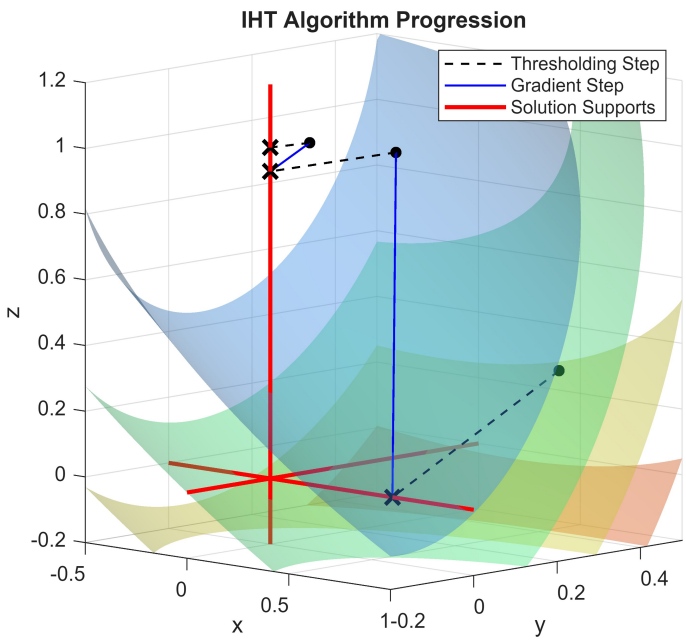

Figure 3: The progression of the iterative hard thresholding algorithm when solving a 2×3 system and searching for a 1-sparse solution.

This scheme closely resembles the standard steepest descent iteration, with the key difference being the hard thresholding step, which enforces sparsity at every iteration. Note that the gradient $\mathbf{r}_i$ cannot be updated using the usual recursive formula from steepest descent, since the thresholding step breaks the Krylov subspace structure. As a result, the gradient must be recomputed directly at each iteration. Furthermore, the thresholding step means that we no longer have a guaranteed reduction in the squared residual at each complete iteration.

3 Considerations, Limitations and Refinement

3.1 Structural Problems

Numerical methods based on gradient descent are often sensitive to the conditioning of the underlying system. If this sensitivity is not controlled, iterations may fail to move decisively toward the solution and instead oscillate, resulting in slow or unreliable convergence. In settings where $m \geq n$, this sensitivity is typically quantified by the condition number of A, or equivalently of $A^\top A$ when it is invertible. A high condition number indicates that the system is ill-conditioned and that gradient-based methods will converge slowly.

When $m < n$, as in compressed sensing, the matrix A cannot have full column rank and $A^\top A$ is necessarily singular. The usual condition number therefore does not apply. Furthermore, in many classical applications, convergence is improved by preconditioning the system to reduce its condition number. A notable distinction in compressed sensing is that the sensing matrix A is not given to us but chosen in advance. Since preconditioning is unnecessary in this setting, the goal is instead to construct A so that it provides a framework for rapid convergence.

3.1.1 Restricted Isometry Property (RIP) and Coherent Columns

An analogous measure to a condition number exists for underdetermined systems, the *Restricted Isometry Property* (RIP) which is applied to sparse vectors. The RIP's definition is as follows.

Restricted Isometry Property (RIP) Let $A \in \mathbb{R}^{m\times n}$ and let $k \in \{1, \ldots, n\}$. If there exists $\delta_k \in (0, 1)$ such that for all k-sparse vectors $\mathbf{x}_k \in \mathbb{R}^n$,

$$(1 - \delta_k)\|\mathbf{x}_k\|_2^2 \;\leq\; \|A\mathbf{x}_k\|_2^2 \;\leq\; (1 + \delta_k)\|\mathbf{x}_k\|_2^2,$$

then A is said to satisfy the k-restricted isometry property with restricted isometry constant δ_k.

A matrix satisfying the k-RIP with a small constant δ_k preserves the Euclidean norm of all k-sparse vectors up to a small distortion. This is directly analogous to the interpretation of a well-conditioned matrix, but with the comparison restricted to sparse vectors rather than all of $\mathbb{R}^n$. In particular, gradient-based sparse recovery methods exhibit much faster and more stable convergence when the sensing matrix satisfies the k-RIP with a low value of δ_k.

Since we have the option to choose A, we can consider constructions of A that satisfy a k-RIP with a low value of δ_k. One common choice of such a constructor is sub-Gaussian random matrices.

Sub-Gaussian Matrix Let $A \in \mathbb{R}^{m\times n}$. The matrix A is called sub-Gaussian if each entry satisfies

$$A_{ij} \sim \mathcal{N}\left(0, \frac{1}{m}\right).$$

Sub-Gaussian random matrices satisfy the k-RIP with high probability provided that

$$m \gtrsim \delta_k^{-2}\, k \log\left(\frac{n}{k}\right),$$

which ensures that sparse vectors are nearly preserved under the action of A. Using such matrices significantly increases the likelihood of successful sparse recovery.

There is a further advantage in using sub-Gaussian matrices: their columns typically exhibit low coherence. Coherence measures how similar two columns are, and is defined by

$$\mu(A) = \max_{i\neq j} \frac{\|\mathbf{e}_i^\top A^\top A \mathbf{e}_j^\top\|_2}{\|A\mathbf{e}_i^\top\|_2\, \|A\mathbf{e}_j^\top\|_2}.$$

If two columns of A, index by i and j say, have high coherence, then $A\mathbf{e}_i$ and $A\mathbf{e}_j$ are nearly indistinguishable (after normalisation). This is a serious obstacle when identifying the correct support of a sparse vector, since highly correlated columns make it difficult for thresholding steps to separate one index from another. Sub-Gaussian matrices, however, are constructed so that their entries are independent and centred, concentrating their pairwise inner products sharply around zero. As a consequence, with high probability the coherence of a sub-Gaussian matrix is small, further improving the stability of sparse recovery.

3.1.2 Normalised Columns

In some implementations of the Iterative Hard Thresholding (IHT) algorithm, solutions may be systematically biased towards particular supports, even when the sensing matrix A is sub-Gaussian. One mechanism through which this bias can arise is significant variability in the column norms of A, which can distort the gradient updates and influence which coordinates are retained after thresholding.

The following result provides a simple explanation for why columns of differing magnitudes can bias the support of the recovered solution.

Theorem 2. *Let $\hat{\mathbf{r}}_k = \mathbf{b} - A\mathbf{x}_k$ denote the residual after the k-th iteration of steepest gradient descent, and let $\mathbf{r}_k = A^\top \hat{\mathbf{r}}_k$ be the corresponding gradient. For indices $i, j \in \{1, \ldots, n\}$ with $i \neq j$, suppose that*

$$\cos\theta_{ik} \approx \cos\theta_{jk},$$

where θ_{lk} denotes the angle between $A\mathbf{e}_l$ and $\hat{\mathbf{r}}_k$. If

$$\|A\mathbf{e}_i\|_2 \ll \|A\mathbf{e}_j\|_2,$$

then the magnitude of the update in the i-th coordinate satisfies

$$|\Delta x_{k,i}| \ll |\Delta x_{k,j}|.$$

Proof. For any index $l \in \{1, \ldots, n\}$, the update in the l-th coordinate of $\mathbf{x}$ produced by a steepest descent step is

$$\Delta x_{k,l} = \alpha_k(\mathbf{e}_l^\top \mathbf{r}_k) = \alpha_k(\mathbf{e}_l^\top A^\top \hat{\mathbf{r}}_k) = \alpha_k(A\mathbf{e}_l)^\top \hat{\mathbf{r}}_k.$$

Writing this inner product in terms of norms and angles gives

$$\Delta x_{k,l} = \alpha_k \|A\mathbf{e}_l\|_2 \|\hat{\mathbf{r}}_k\|_2 \cos\theta_{lk}.$$

If $\cos\theta_{ik} \approx \cos\theta_{jk}$ and $\|A\mathbf{e}_i\|_2 \ll \|A\mathbf{e}_j\|_2$, the conclusion follows immediately. □

This result shows that columns of A with larger Euclidean norms contribute more strongly to the gradient vector. As a consequence, gradient-based iterations tend to produce larger updates in the coordinates corresponding to larger-norm columns. In practical terms, columns of A with larger norms tend to attract larger coefficients in $\mathbf{x}$.

Since the hard thresholding step retains the largest-magnitude entries of $\mathbf{x}$, any systematic bias in the gradient updates undermines the algorithm's ability to correctly identify the true support of the sparse solution. If the variability in column norms is sufficiently large, this bias can dominate the effect of the data $\mathbf{b}$, leading to consistent selection of incorrect supports.

Because sub-Gaussian matrices are random with independent entries, their columns naturally exhibit variability in norm. Understanding when this variability is negligible and when it is significant helps determine whether column normalisation is beneficial.

Theorem 3. *Let $A \in \mathbb{R}^{m\times n}$ be a sub-Gaussian matrix with entries $A_{ij} \sim \mathcal{N}(0, 1/m)$. Then for any column $A\mathbf{e}_j$,*

$$\mathbb{E}\big[\|A\mathbf{e}_j\|_2^2\big] = 1, \qquad \mathrm{Var}\big(\|A\mathbf{e}_j\|_2^2\big) = \frac{2}{m}.$$

Proof. Fix $j \in \{1, \ldots, n\}$. Then

$$\|A\mathbf{e}_j\|_2^2 = \sum_{i=1}^{m} A_{ij}^2.$$

Define $Z_{ij} = \sqrt{m}A_{ij}$, so that each $Z_{ij} \sim \mathcal{N}(0, 1)$ independently. Then

$$m\|A\mathbf{e}_j\|_2^2 = \sum_{i=1}^{m} Z_{ij}^2 \sim \chi_m^2.$$

The chi-squared distribution with m degrees of freedom has mean m and variance $2m$, and the result follows by rescaling. □

This shows that column norms concentrate sharply around one as m increases. Consequently, for large m the effects of non-normalised columns are typically mild, whereas for small m the variability can be substantial and column normalisation may significantly improve recovery performance.

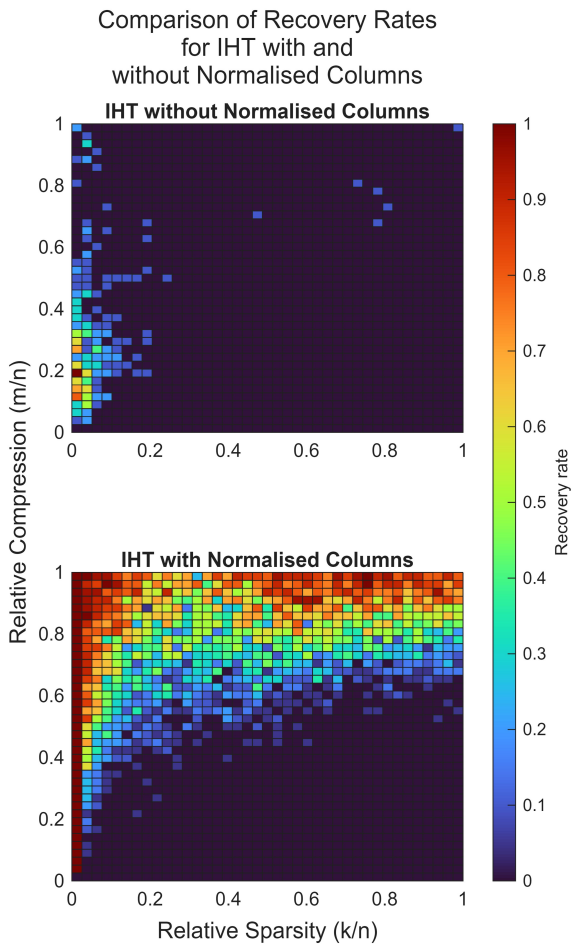

Figure 4: Recovery rates for the IHT with and without columns. The sensitivity matrix is sub-Gaussian, the recoverable solution is a sparse standard normal vector and the initial guess is the origin.

Before we can weigh up whether column normalisation is beneficial, it is important to understand how column scaling affects the Restricted Isometry Property.

Theorem 4. *Let $A \in \mathbb{R}^{m\times n}$ satisfy the k-restricted isometry property with constant $\delta_k \in (0,1)$. Let $\hat{A}$ be obtained by scaling a single column of A, indexed by j, by a factor $c > 0$. Then $\hat{A}$ satisfies the k-restricted isometry property with constant*

$$\hat{\delta}_k = \max\left(1 - (1-\delta_k)\min(1,c^2),\ (1+\delta_k)\max(1,c^2) - 1\right).$$

Proof. Fix $j \in \{1,\dots,n\}$. Define the diagonal matrix

$$D = \operatorname{diag}(1,\dots,1,c,1,\dots,1),$$

where the entry c appears in the (j,j)-th position. Then $\hat{A} = AD$ corresponds to scaling the j-th column of A by c.

Let $\mathbf{x} \in \mathbb{R}^n$ be an arbitrary k-sparse vector. Since D is diagonal, if j isn't in the support of $\mathbf{x}_k$ then $D\mathbf{x} = \mathbf{x}$ and if j is in the support, the vector $D\mathbf{x}$ differs from $\mathbf{x}$ only in its j-th entry which is scaled by c.In particular, the support of $D\mathbf{x}$ is contained within the support of $\mathbf{x}$, and hence $D\mathbf{x}$ is also k-sparse.

Now observe that

$$\|\hat{A}\mathbf{x}\|_2^2 = \|AD\mathbf{x}\|_2^2 = \|A(D\mathbf{x})\|_2^2.$$

Applying the k-RIP of A to the k-sparse vector $D\mathbf{x}$ gives

$$(1-\delta_k)\|D\mathbf{x}\|_2^2 \le \|\hat{A}\mathbf{x}\|_2^2 \le (1+\delta_k)\|D\mathbf{x}\|_2^2.$$

Moreover,

$$\|D\mathbf{x}\|_2^2 = \sum_{i\neq j} x_i^2 + c^2 x_j^2,$$

which implies

$$\min(1,c^2)\|\mathbf{x}\|_2^2 \le \|D\mathbf{x}\|_2^2 \le \max(1,c^2)\|\mathbf{x}\|_2^2.$$

Combining these bounds yields

$$(1-\delta_k)\min(1,c^2)\|\mathbf{x}\|_2^2 \le \|\hat{A}\mathbf{x}\|_2^2 \le (1+\delta_k)\max(1,c^2)\|\mathbf{x}\|_2^2.$$

Rewriting the constants in the form $(1-\hat{\delta}_k)$ and $(1+\hat{\delta}_k)$ gives

$$\hat{\delta}_k = \max\left[1 - (1-\delta_k)\min(1,c^2),\ (1+\delta_k)\max(1,c^2) - 1\right],$$

which completes the proof. □

In particular, if $c^2 = 1+\varepsilon$ with $\varepsilon \ll 1$ and $\delta_k < 1$, then

$$\hat{\delta}_k = \delta_k + O(|\varepsilon|).$$

Since the column norms of sub-Gaussian matrices concentrate tightly around one, column normalisation perturbs the restricted isometry constant only slightly and therefore does not destroy the RIP. The massive effect column normalisation has on recovery rates is depicted in Figure 4.

Finally, it is useful to quantify when column normalisation is necessary. To do so, we require two tolerances: (1) a magnitude tolerance α describing how far from unit norm a column may be without inducing bias; and (2) a probability tolerance β describing the minimum acceptable probability that all columns satisfy this magnitude bound. The following theorem provides an approximate inequality relating these quantities for sub–Gaussian matrices.

Theorem 5. *Let $A \in \mathbb{R}^{m\times n}$ be a sub–Gaussian matrix. Then, for size tolerance $\alpha > 0$ and probability tolerance $\beta \in (0,1)$, the inequality*

$$\Phi\left(\frac{(1+\alpha)^{1/3} - \mu}{\sigma}\right) - \Phi\left(\frac{(1-\alpha)^{1/3} - \mu}{\sigma}\right) \lesssim \beta^{1/n},$$

where

$$\mu = 1 - \frac{2}{9m}, \qquad \sigma = \sqrt{\frac{2}{9m}},$$

approximates the condition under which the probability that every column of A has squared Euclidean norm within α of 1 falls below β.

Proof. Since A is sub–Gaussian, its entries are independent, and hence the squared Euclidean norms of distinct columns are independent random variables. Therefore, for any indices $i, j \in \{1, \ldots, n\}$, with $i \neq j$,

$$P\big(\big|\|A\mathbf{e}_i\|_2^2 - 1\big| < \alpha \cap \big|\|A\mathbf{e}_j\|_2^2 - 1\big| < \alpha\big) = P\big(\big|\|A\mathbf{e}_i\|_2^2 - 1\big| < \alpha\big)\ P\big(\big|\|A\mathbf{e}_j\|_2^2 - 1\big| < \alpha\big)\,,$$

and these probabilities are identical because each column shares the same distribution. Thus, the probability that all columns satisfy the bound is

$$\big[P\big(\big|\|A\mathbf{e}_k\|_2^2 - 1\big| < \alpha\big)\big]^n\,,$$

where k denotes an arbitrary column index.

We therefore require

$$P\big(\big|\|A\mathbf{e}_k\|_2^2 - 1\big| < \alpha\big) = \beta^{1/n}.$$

The left–hand probability equals

$$P(\|A\mathbf{e}_k\|_2^2 < 1 + \alpha) - P(\|A\mathbf{e}_k\|_2^2 < 1 - \alpha),$$

which depends on the CDF of a scaled chi–squared distribution. We therefore adopt the Wilson–Hilferty cube–root approximation, which is accurate for small degrees of freedom.

Let $m\|A\mathbf{e}_k\|_2^2 \sim \chi_m^2$. Then for any critical value γ,

$$P(\|A\mathbf{e}_k\|_2^2 < \gamma) = P(m\|A\mathbf{e}_k\|_2^2 < m\gamma).$$

Applying the Wilson–Hilferty approximation,

$$\left(\frac{m\|A\mathbf{e}_k\|_2^2}{m}\right)^{1/3} \approx N\left(1 - \frac{2}{9m}, \frac{2}{9m}\right).$$

Thus,

$$P(\|A\mathbf{e}_k\|_2^2 < \gamma) \approx P\left(\frac{\|A\mathbf{e}_k\|_2^{2/3} - \mu}{\sigma} < \frac{\gamma^{1/3} - \mu}{\sigma}\right) = \Phi\left(\frac{\gamma^{1/3} - \mu}{\sigma}\right),$$

where $\mu = 1 - 2/(9m)$, $\sigma^2 = 2/(9m)$, and Φ is the standard normal CDF.

Substituting $\gamma = 1 + \alpha$ and $\gamma = 1 - \alpha$ yields the claimed approximation. □

3.2 Cycles

Despite the potential existence of a unique sparse solution, the IHT algorithm often struggles to converge to it. This is largely because IHT may fail to identify the correct support of the sparse solution. Iterations can become locked into an incorrect support from which they cannot escape, causing the squared residual to stagnate and preventing convergence to within a prescribed tolerance. If the algorithm begins to cycle, then it will fail to recover the sparse solution without further intervention.

3.2.1 Cycle Classification

There are two types of cycles that can occur when searching for a sparse solution:

Type-I Cycle: The iterations jump back and forth over a repeating loop of supports. This behaviour prevents the algorithm from identifying the correct support and results in a stagnating residual. For a well-conditioned sensing matrix A, this almost always occurs when the stepsize is too large. In the IHT framework, where the stepsize is optimised along the current search direction, an excessively large stepsize is unlikely; for this reason, Type-I cycles tend to arise in fixed–stepsize algorithms, and the risk diminishes sharply when a well-adapted stepsize is used.

Type-II Cycle: Iterations remain in the same support without significant reduction in the residual. These cycles occur when the thresholding step substantially counteracts the gradient step and prevents the iterates from escaping a particular sparse support, even though the residual is stagnating. Geometrically, this corresponds to the gradient vector being normal to the active sparse solution space, or equivalently, the sparse solution space being tangential to a residual contour. A larger stepsize in this situation allows a gradient step large enough not to be undone by thresholding and therefore increases the likelihood of escaping the current support.

3.2.2 Cycle Detection

Cycles can be detected by monitoring both the progression of the residual and the support of the current solution. Doing so is important when applying modifications designed to break such behaviour. In practice, rather than continually storing and comparing supports, it is often more efficient to monitor only the residual and trigger a cycle–detection routine once a plateau is observed. At that point, both the residual and support are analysed to classify a possible cycle and determine how to resolve it.

3.2.3 Breaking the Cycle

There are many methods to overcome the problem of cycles, varying in complexity and computational cost. These approaches may be used in combination, beginning with inexpensive interventions and escalating to more sophisticated mechanisms if necessary. A robust collection of cycle–breaking tools significantly increases the likelihood of sparse recovery.

Manual Scaling
The simplest method is a manual rescaling of the stepsize. In the case of a Type-I cycle, reducing the stepsize may dampen oscillation between supports. For a Type-II cycle, increasing the stepsize may provide sufficient movement to escape a locked support. A typical implementation introduces a parameter $\beta > 1$: when a Type-II cycle is detected, the stepsize μ_i is replaced by $\beta\mu_i$ in an attempt to escape the current support.

Adaptive Sparsity
A more nuanced strategy is to temporarily increase the sparsity level and then taper it back down. Although not guaranteed to succeed, this widens the feasible space of sparse candidates to potentially include the true minimiser. The sparsity parameter is then reduced again, with the hope that the iterates settle into the correct support. The precise evolution of the sparsity level is problem–dependent and left to the user.

Normalised Iterative Thresholding (NIHT)
A more integrated approach is the normalised iterative hard thresholding (NIHT) algorithm. Unlike the preceding adaptations, NIHT does not rely on cycle detection; instead, it incorporates a structural modification which promotes a change of support when the current support is incorrect, and inhibits such changes when the correct support has been found. The NIHT iteration differs from IHT only in the choice of search direction. In IHT, the direction is

$$\mathbf{r}_i = A^\top(\mathbf{b} - A\mathbf{x}_i),$$

whereas NIHT replaces this by its k–sparse approximation,

$$\mathbf{r}_{k,i} = \mathcal{H}_k(A^\top(\mathbf{b} - A\mathbf{x}_i)).$$

This subtle change preserves the main advantages of IHT—particularly the steepest–descent character of the gradient—while mitigating several of its weaknesses.

When the correct support has been found, thresholding the gradient has negligible effect, because the support of $A^\top(\mathbf{b} - A\mathbf{x}_i)$ naturally aligns with the support of the solution. In this regime, $\mathbf{r}_i \approx \mathcal{H}_k(\mathbf{r}_i)$ and the NIHT update behaves nearly identically to IHT, continuing to descend to the solution over the correct support.

In contrast, when the support is incorrect, a key observation arises from the monotonicity of RIP constants: for $k_1 \leq k_2$, if A satisfies the k_1–RIP with constant δ_{k_1} and the k_2–RIP

with constant δ_{k_2}, then $\delta_{k_1} \leq \delta_{k_2}$. Consequently, sparser vectors tend to undergo less distortion under A than denser ones. Applied to the NIHT stepsize, this motivates the comparison

$$\frac{\alpha_i^{\text{NIHT}}}{\alpha_i^{\text{IHT}}} = \frac{\left[\frac{\|\mathcal{H}_k(\mathbf{r}_i)\|_2^2}{\|A\mathcal{H}_k(\mathbf{r}_i)\|_2^2}\right]}{\left[\frac{\|\mathbf{r}_i\|_2^2}{\|A\mathbf{r}_i\|_2^2}\right]} = \frac{\left[\frac{\|A\mathbf{r}_i\|_2^2}{\|\mathbf{r}_i\|_2^2}\right]}{\left[\frac{\|A\mathcal{H}_k(\mathbf{r}_i)\|_2^2}{\|\mathcal{H}_k(\mathbf{r}_i)\|_2^2}\right]}.$$

Since $\mathcal{H}_k(\mathbf{r}_i)$ is sparser than $\mathbf{r}_i$, RIP monotonicity suggests

$$\frac{\|A\mathbf{r}_i\|_2^2}{\|\mathbf{r}_i\|_2^2} \gg \frac{\|A\mathcal{H}_k(\mathbf{r}_i)\|_2^2}{\|\mathcal{H}_k(\mathbf{r}_i)\|_2^2}, \quad \text{when } n \gg k,$$

and therefore $\alpha_i^{\text{NIHT}} \gg \alpha_i^{\text{IHT}}$ in this regime, with equality emerging as soon as the correct support is found.

To illustrate the implications for cycling, consider a Type-I cycle encountered at $\mathbf{x}_k$. Let $\mathbf{m}_k$ be an arbitrary point on the active subspace. Then $\mathbf{x}_k - \mathbf{m}_k$ lies in the same subspace, and Type-I behaviour implies

$$\mathbf{r}^\top(\mathbf{x}_k - \mathbf{m}_k) = 0,$$

i.e. $\mathbf{r}$ is normal to the active sparse subspace. Since $\mathbf{m}_k$ is arbitrary, the supports of $\mathbf{x}_k - \mathbf{m}_k$ and $\mathbf{r}$ are disjoint, and similarly

$$\text{supp}(\mathbf{x}_k) \cap \text{supp}(\mathcal{H}_k(\mathbf{r})) = \emptyset.$$

Thus, in the NIHT update

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha_i^{\text{NIHT}}\mathcal{H}_k(\mathbf{r}_i),$$

the existing nonzero entries of $\mathbf{x}_i$ remain unchanged, while entries outside the current support are boosted—amplified further by the larger stepsize. This mechanism actively promotes support switching and therefore breaks the cycle. The improvement NIHT offers over IHT is illustrated in Figure 5.

Finally, we note a general trend in Figure 4: smaller m and larger k yield more recoverable solutions. This reflects an intuitive reality: more aggressive compression (smaller m) obscures recoverability, and demanding a more accurate sparse representation (larger k) increases the difficulty of exact recovery. It can in fact be shown that when $k\log(\frac{k}{n}) \lesssim m$ there is a high chance of successful recovery.

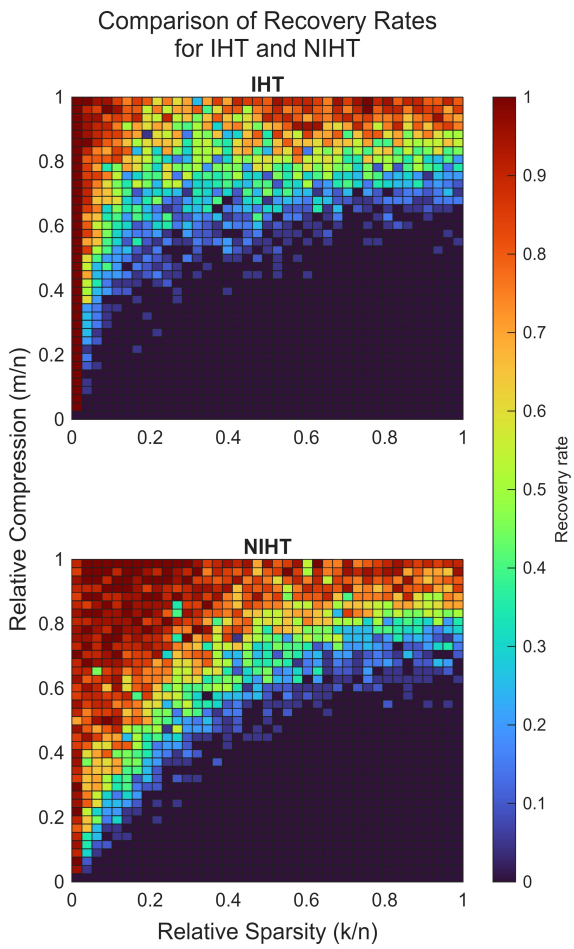

Figure 5: Recovery rates for IHT and NIHT with normalised columns. The sensing matrix is sub–Gaussian, the target solution is a sparse standard normal vector, and the initial guess is the origin.

3.3 Initial Guesses

As with many iterative solution methods, the quality of the initial guess can significantly influence the probability of successful recovery. This naturally raises the question: how can useful initial guesses be generated? A common approach is to initialise with a random vector whose entries are independent, typically with zero mean and adjustable variance. In the context of compressed sensing, however, such random initialisations can result in unstable support selection, leading the algorithm toward cycling behaviour that is costly to resolve. Ideally, the initial guess should encode some information about the true support of the sparse solution, thereby guiding the algorithm along a productive trajectory from the outset. Some implementations of IHT and NIHT therefore construct high-quality initial guesses using greedy search algorithms that identify promising support candidates prior to thresholding, although this comes with additional computational expense.

A practical intermediate strategy leverages the coherence of the columns of A with the data vector $\mathbf{b}$. Intuitively, columns whose images under A are most aligned with $\mathbf{b}$ are more likely to index the true support of the sparse solution. Indeed, if the sparsity level were 1, this identification would be exact. Recall that if the columns of A have been normalised, the coherence between a column $A\mathbf{e}_k$ and $\mathbf{b}$ is given by

$$\text{coherence}(A\mathbf{e}_k, \mathbf{b}) = \frac{(A\mathbf{e}_k)^\top \mathbf{b}}{\|A\mathbf{e}_k\|_2 \, \|\mathbf{b}\|_2} = \frac{(A\mathbf{e}_k)^\top \mathbf{b}}{\|\mathbf{b}\|_2}.$$

Thus, $(A\mathbf{e}_k)^\top \mathbf{b}$ provides a uniformly scaled indicator of coherence strength.

Now consider the first iteration of IHT when the initial guess is the origin, $\mathbf{x}_0 = \mathbf{0}$:

$$\mathbf{x}_1 = \mathcal{H}_k\big(\mathbf{x}_0 + \alpha_0 A^\top(\mathbf{b} - A\mathbf{x}_0)\big) = \mathcal{H}_k(\alpha_0 A^\top \mathbf{b}) = \alpha_0 \, \mathcal{H}_k(A^\top \mathbf{b}).$$

Using NIHT in place of IHT produces an update of the same form, differing only by the stepsize. Hence both approaches select the same support at the first iteration. More precisely, the k nonzero entries of $\mathbf{x}_1$ correspond to the indices of the k largest (in magnitude) entries of $A^\top \mathbf{b}$, and we observe that

$$\mathbf{e}_k^\top \mathbf{x}_1 = \begin{cases} \alpha_0 (A\mathbf{e}_k)^\top \mathbf{b} = \alpha_0 \|\mathbf{b}\|_2 \, \text{coherence}(A\mathbf{e}_k, \mathbf{b}), & \text{if the } k\text{-th column of } A \text{ has one of the } k \text{ largest} \\ & \text{(in magnitude) coherences with } \mathbf{b}, \\ 0, & \text{otherwise.} \end{cases}$$

Thus, after a single iteration starting from the origin, the algorithm identifies the support consisting of the k most coherent columns of A with respect to $\mathbf{b}$, while the nonzero values provide a scaled ranking of their relative importance. For this reason, the origin is a widely used and computationally efficient initialisation: it yields an informative update after one iteration at negligible cost.

In situations where the existence of a unique sparse solution is assured, but a chosen initialisation fails to recover it, a final option is to employ parameter continuation. Figure 4 shows clearly that recovery rates tend to increase for smaller k and larger m. Since m reflects the level of compression, it is typically fixed. The sparsity level k, however, may be adjusted. Although sparse solutions are discontinuous with respect to k-and hence continuation cannot be applied in the classical sense-one may first recover a solution for a smaller, more favourable sparsity level. The resulting support information can then be used to initialise the recovery problem with a larger k. Because the discontinuity tends to be most severe for small k, continuation is most effective when the relative increment $\frac{\Delta k}{k}$ is small.

4 Conclusion

The code for the header files for the random generator, vector class, matrix class as well as the main body of code is given within appendix 5.1, 5.2, 5.3 and 5.3 respectively. Compressed sensing provides a powerful framework for exploiting structure in large-scale data. Across telecommunications, digital imaging, medical diagnostics, and remote sensing, it enables substantial reductions in the volume of data that must be acquired or transmitted, while still permitting high-quality reconstruction. By capitalising on the sparsity or compressibility inherent in many real-world signals, compressed sensing replaces exhaustive measurement with a carefully designed encoding that preserves essential information.

Thresholding algorithms form an important class of reconstruction methods within this framework. Their appeal lies in their simplicity, low computational cost, and scalability to high dimensions, making them particularly attractive in large-scale or resource-constrained settings. In this project, we formulated these algorithms within the context of linear inverse problems, showing how they arise naturally from steepest descent methods augmented by hard sparsity constraints. We also established theoretical conditions under which sparse solutions are unique, ensuring that the recovery problem is well posed when the sensing matrix satisfies appropriate structural properties.

A key advantage of thresholding methods is their flexibility. We demonstrated how random sensing matrices, particularly sub-Gaussian constructions, provide favourable recovery guarantees

by satisfying the Restricted Isometry Property with high probability and exhibiting low column coherence. We showed that column normalisation, while leaving the RIP essentially unchanged, can significantly improve recovery performance by mitigating biases in gradient updates and clarifying distinctions between candidate supports—an effect that becomes increasingly important as compression becomes more aggressive or problem dimensions grow.

Despite these strengths, thresholding algorithms are not without limitations. Their iterative nature makes them susceptible to cycling behaviour, in which iterations either oscillate between supports or stagnate within an incorrect support, preventing further reduction of the residual. We classified these cycles, explained their geometric origins, and discussed practical detection strategies. Several remedies were analysed, ranging from manual stepsize scaling and adaptive sparsity to more integrated approaches such as Normalised Iterative Hard Thresholding (NIHT). By thresholding the search direction itself, NIHT promotes support changes when the current support is incorrect, while preserving descent behaviour once the correct support has been identified. Additional robustness can be achieved through informed initialisation strategies, including warm starts based on coherence, lightweight greedy searches, or continuation in the sparsity parameter.

It is important to place thresholding algorithms within the broader ecosystem of sparse recovery methods. Convex ℓ_1-minimisation techniques, such as Basis Pursuit and LASSO, offer strong theoretical guarantees and robustness to noise, but at a significantly higher computational cost. Bayesian approaches provide probabilistic uncertainty quantification and can incorporate prior information, though they are often expensive and model-dependent. Greedy algorithms, such as Orthogonal Matching Pursuit and CoSaMP, offer a middle ground, with improved stability over basic thresholding but increased complexity per iteration. In practice, heavily modified thresholding algorithms are most effective when fast approximate recovery is required and occasional failure can be tolerated or mitigated through restarts. In contrast, high-stakes applications—such as medical imaging or safety-critical sensing—may favour more conservative methods with stronger worst-case guarantees, even at the expense of computational efficiency, due to the severe consequences of reconstruction errors.

Further research is required to fully understand the convergence behaviour and failure modes of sparse recovery algorithms, particularly in regimes where unique sparse solutions are known to exist but are difficult to compute in practice. Improved theoretical characterisations of cycling, sharper recovery guarantees for adaptive algorithms, and hybrid methods that combine the efficiency of thresholding with the robustness of convex or Bayesian approaches remain promising directions. As compressed sensing continues to transition from theory to practice, addressing these challenges will be essential to building trust in its deployment, especially in domains where even small reconstruction errors may have significant real-world consequences.

5 Appendix

5.1

```
#ifndef RAND_UTILS_H
#define RAND_UTILS_H

#include <cmath>
#include <cstdlib>
#include <ctime>

inline double rand_normal()
{
    static bool seeded = false;
    if (!seeded) {
        std::srand(static_cast<unsigned>(std::time(nullptr))); // seed once
        seeded = true;
    }

    static const double pi = 3.141592653589793238;
    double u = 0;
```

```
    while (u == 0) // loop to ensure u nonzero, for log
    {
        u = rand() / static_cast<double>(RAND_MAX);
    }
    double v = rand() / static_cast<double>(RAND_MAX);
    return std::sqrt(-2.0 * std::log(u)) * std::cos(2.0 * pi * v);
}

#endif
```

5.2

```
#ifndef MVECTOR_H // the 'include guard'
#define MVECTOR_H // see C++ Primer Sec. 2.9.2

#include <vector>
#include <iostream>
#include <algorithm>
#include <cmath>
#include <numeric>
#include <random>
#include <cstdlib>
#include "rand_norm.h"
#include <ctime>

// Class that represents a mathematical vector
class MVector
{
public:
   // constructors
   MVector() {}
   explicit MVector(int n) : v(n) {}
   MVector(int n, double x) : v(n, x) {}
   MVector(std::initializer_list<double> l) : v(l) {}

   // access element (lvalue) (see example sheet 5, q5.6)
   double &operator[](int index)
   {
      return v[index];
   }

   // access element (rvalue) (see example sheet 5, q5.7)
   double operator[](int index) const {
      return v[index];
   }

   int size() const { return v.size(); } // number of elements

   // add data to your vector
   void push_back(double x){v.push_back(x);}

   // infinity norm
   double LInfNorm() const
   {
      double maxAbs = 0;
      std::size_t s = size();
      for (int i=0; i<s; i++)
      {
         maxAbs = std::max(std::abs(v[i]), maxAbs);
      }
```

```
      return maxAbs;
   }

   // L2 norm
   double L2Norm() const
   {
      double sum = 0;
      std::size_t s = size();
      for (int i=0; i<s; i++)
      {
         sum+=v[i]*v[i];
      }
      return std::sqrt(sum);
   }

   auto begin() { return v.begin(); }

   auto end()  { return v.end(); }

   auto begin() const { return v.begin(); }

   auto end() const { return v.end(); }

   // Threshold
   void Threshold(int k)
   {
      MVector w(v.size(),0);
        auto itmin = std::min_element(v.begin(), v.end());
        int indexmin = std::distance(v.begin(), itmin);
        auto itmax = std::max_element(v.begin(), v.end());
        int indexmax = std::distance(v.begin(), itmax);
        for(int i=0;i<k;i++)
        {
            if(std::abs(*itmax)>std::abs(*itmin))
            {
                w[indexmax]=*itmax;
                v[indexmax]=0;
                itmax = std::max_element(v.begin(), v.end());
                indexmax = std::distance(v.begin(), itmax);
            }
            else
            {
                w[indexmin]=*itmin;
                v[indexmin]=0;
                itmin = std::min_element(v.begin(), v.end());
                indexmin = std::distance(v.begin(), itmin);
            }
        }
      *this=w;
   }

   // sparse initialiser
   void initialise_sparse_normal(int k)
   {
      if (k > size())
      {
         std::cout<<"invalid argument"<<std::endl;
         return;
      }
      // Step 2: generate indices [0..size()-1]
      std::vector<int> indices(size());
```

```
      std::iota(indices.begin(), indices.end(), 0);
      // Step 3: shuffle them
      std::mt19937 gen(static_cast<unsigned>(std::time(nullptr)));
      std::shuffle(indices.begin(), indices.end(), gen);
      // Step 4: set k random positions using rand_normal()
      for (int i = 0; i < k; ++i) {
         v[indices[i]] = rand_normal();
      }
   }

private:
   std::vector<double> v;
};

//1.2.1

inline MVector operator*(const double& lhs, const MVector& rhs)
{
   MVector temp = rhs;
   for (int i=0; i<temp.size(); i++) temp[i]*=lhs;
   return temp;
}

inline MVector operator*(const MVector& rhs, const double& lhs)
{
   MVector temp = rhs;
   for (int i=0; i<temp.size(); i++) temp[i]*=lhs;
   return temp;
}

inline MVector operator/(const MVector& rhs, const double& lhs)
{
   MVector temp = rhs;
   for (int i=0; i<temp.size(); i++) temp[i]/=lhs;
   return temp;
}

inline MVector operator+(const MVector& lhs, const MVector& rhs)
{
   if (lhs.size()!=rhs.size())
   {
      std::cout<<"incompatible vector size"<<std::endl;
      return MVector();
   }
   MVector temp = rhs;
   for (int i=0; i<temp.size(); i++) temp[i]=lhs[i]+rhs[i];
   return temp;
}

inline MVector operator-(const MVector& lhs, const MVector& rhs)
{
   MVector temp = rhs;
   for (int i=0; i<temp.size(); i++) temp[i]=lhs[i]-rhs[i];
   return temp;
}

inline double dot(const MVector& lhs, const MVector& rhs)
{
   if (lhs.size()!=rhs.size())
   {
      std::cout<<"incompatible vectors"<<std::endl;
```

```
      return 1e6;
   }
   double sum=0;
   for(int i=0;i<lhs.size();i++)
   {
      sum+=lhs[i]*rhs[i];
   }
   return sum;
}

std::ostream& operator<<(std::ostream& os, const MVector& v)
{
   int n = v.size();
   os << "("<<v[0];
   for(int i=1; i<n;i++) os<<","<<v[i];
   os << ")";
   return os;
}

#endif
```

5.3

```
#ifndef MMATRIX_H // the 'include guard'
#define MMATRIX_H

#include <vector>
#include <cstdlib>
#include <cmath>
#include "rand_norm.h"




// Class that represents a mathematical matrix
class MMatrix
{
public:
   // constructors
   MMatrix() : nRows(0), nCols(0) {}
   MMatrix(int n, int m, double x = 0) : nRows(n), nCols(m), A(n * m, x) {}

   // set all matrix entries equal to a double
   MMatrix &operator=(double x)
   {
      for (unsigned i = 0; i < nRows * nCols; i++) A[i] = x;
      return *this;
   }

   // access element, indexed by (row, column) [rvalue]
   double operator()(int i, int j) const
   {
      return A[j + i * nCols];
   }

   // access element, indexed by (row, column) [lvalue]
   double &operator()(int i, int j)
   {
      return A[j + i * nCols];
```

```
   }

   // size of matrix
   int Rows() const { return nRows; }
   int Cols() const { return nCols; }

   void initialise_normal()
   {
      for(int i=0; i<nRows;i++)
      {
         for(int j=0; j<nCols;j++)
         {
            A[j + i * nCols]=rand_normal()/nRows;
         }
      }

   }

   void normalise_columns()
{
    for(int j = 0; j < nCols; j++)
    {
        // compute column norm
        double norm = 0.0;
        for(int i = 0; i < nRows; i++)
        {
            double val = A[j + i*nCols];
            norm += val * val;
        }
        norm = std::sqrt(norm);

        // avoid division by zero
        if(norm > 0.0)
        {
            for(int i = 0; i < nRows; i++)
            {
                A[j + i*nCols] /= norm;
            }
        }
    }
}


private:
   unsigned int nRows, nCols;
   std::vector<double> A;
};

inline MVector operator*(const MMatrix& A, const MVector& x)
{
   if (x.size()!=A.Cols())
   {
      std::cout<<"incompatible operator"<<std::endl;
      return {0};
   }
   MVector v(A.Rows());
   for (int i=0; i<A.Rows(); i++)
   {
      v[i]=0;
      for(int j=0; j<A.Cols();j++)
      {
```

```
            v[i]+=A(i,j)*x[j];
         }
      }
      return v;
   }

   inline MMatrix operator*(const MMatrix& A, const MMatrix& B)
   {
      if(A.Cols() !=B.Rows())
      {
         std::cout<<"incompatible matrix operation"<<std::endl;
         MMatrix C(0,0,0);
         return C;
      }
      MMatrix D(A.Rows(),B.Cols(),0);
      for(int i=0;i<A.Rows();i++)
      {
         for(int j=0; j<B.Cols();j++)
         {
            for(int k=0; k<A.Cols();k++)
            {
               D(i,j)+=A(i,k)*B(k,j);
            }
         }
      }
      return D;
   }

   inline MMatrix Transpose(const MMatrix& A)
   {
      MMatrix B(A.Cols(), A.Rows(),0);
      for(int i=0; i<A.Cols(); i++)
      {
         for(int j=0; j<A.Rows(); j++)
         {
            B(i,j)=A(j,i);
         }
      }
      return B;
   }

   std::ostream& operator<<(std::ostream& os, const MMatrix& A)
   {
      os.width(1);
      os << "(";
      os.width(9);
      os.precision(9);
      os<<A(0,0);
      os<<",";

      for(int i=0;i<A.Rows();i++)
      {
         for(int j=0;j<A.Cols();j++)
         {
            if (i==0 && j==0) continue;
            os.width(10);
            os.precision(10);
            os<<A(i,j);
            if(j<A.Cols()-1) os<<",";
         }
         if(i<A.Rows()-1) os<<std::endl;
```

```
    }
    os << ")";
    return os;
}

#endif
```

5.4

```
#include <iostream>
#include "mvector.h"
#include "mmatrix.h"
#include <cmath>
#include <string>
#include <fstream>
#include <algorithm>

// for contour plots
void residualcontour(const std::string& filename, const MMatrix& A, const MVector& b, double x0, double x1, double y0, double y1, int nx, int ny)
{
    std::ofstream demofile;
    demofile.open(filename);
    if (!demofile)
    {
        std::cout<<"unable to open file";
        return;
    }
    for(int i=0; i<nx; i++)
    {
        for(int j=0; j<ny; j++)
        {
            MVector x={x0+(i*(x1-x0))/(nx-1),y0+(j*(y1-y0))/(ny-1)};
            demofile<<dot(b-(A*x),b-(A*x))<<" ";
        }
        demofile<<"\n";
    }
    demofile.close();
}

// for 3dcontour plots
void residual3dcontour(const std::string& filename, const MMatrix& A, const MVector& b, double x0, double x1, double y0, double y1, double z0, double z1, int nx, int ny, int nz)
{
    std::ofstream demofile;
    demofile.open(filename);
    if (!demofile)
    {
        std::cout<<"unable to open file";
        return;
    }
    for(int i=0; i<nx; i++)
    {
        for(int j=0; j<ny; j++)
        {
            for(int k=0; k<nz; k++)
            {
                MVector x={x0+(i*(x1-x0))/(nx-1),y0+(j*(y1-y0))/(ny-1),z0+(k*(z1-z0))/(nz-1)};
```

```
                demofile<<dot(b-(A*x),b-(A*x))<<" ";
            }
        }
    }
    demofile.close();
}

// SDLS
int SDLS(const MMatrix& A, const MVector& b, MVector& x,
int maxIterations, double tol)
{
    if (A.Cols()!=x.size() || A.Rows()!=b.size())
    {
        std::cout<<"incompatible arguments"<<std::endl;
        return 0;
    }
    double alpha;
    MVector r=Transpose(A)*(b-A*x);
    for(int iter=0; iter<=maxIterations; iter++)
    {
        std::cout<<x[1]<<" ";
        if (r.L2Norm() < tol)
        {
            return iter;
        }
        MVector q=A*r;
        alpha=dot(r,r)/dot(q,q);
        x=x+alpha*r;
        r=r-alpha*(Transpose(A)*q);
    }
    return 0;
}


//IHT
int IHT(const MMatrix& A, const MVector& b, MVector&x, int k,
int maxIterations, double tol)
{
    if (A.Cols()!=x.size() || A.Rows()!=b.size())
    {
        std::cout<<"incompatible arguments"<<std::endl;
        return 0;
    }
    double alpha;
    MVector r=Transpose(A)*(b-A*x);
    for(int iter=0; iter<=maxIterations; iter++)
    {
        MVector q=A*r;
        alpha=dot(r,r)/dot(q,q);
        x=x+alpha*r;
        x.Threshold(k);
        r=Transpose(A)*(b-A*x);
        if (r.L2Norm() < tol)
        {
            return iter;
        }
    }
    return 0;
}

//NIHT
```

```
int NIHT(const MMatrix& A, const MVector& b, MVector&x, int k,
int maxIterations, double tol)
{
    if (A.Cols()!=x.size() || A.Rows()!=b.size())
    {
        std::cout<<"incompatible arguments"<<std::endl;
        return 0;
    }
    double alpha;
    MVector r=Transpose(A)*(b-A*x);
    for(int iter=0; iter<=maxIterations; iter++)
    {
        r.Threshold(k);
        MVector q=A*r;
        alpha=dot(r,r)/dot(q,q);
        x=x+alpha*r;
        x.Threshold(k);
        r=Transpose(A)*(b-A*x);
        if (r.L2Norm() < tol)
        {
            return iter;
        }
    }
    return 0;
}

// succesful recovery
int successfulrecoveryIHT(const MMatrix& A, const MVector& x, MVector&x0, int k,
int maxIterations, double tol1,double tol2)
{
    MVector b=A*x;
    IHT(A, b, x0, k, maxIterations, tol1);
    if(dot(x-x0, x-x0)/dot(x,x)<tol2) return 1;
    else return 0;
}

// succesful recovery
int successfulrecoveryNIHT(const MMatrix& A, const MVector& x, MVector&x0, int k,
int maxIterations, double tol1,double tol2)
{
    MVector b=A*x;
    NIHT(A, b, x0, k, maxIterations, tol1);
    if(dot(x-x0, x-x0)/dot(x,x)<tol2) return 1;
    else return 0;
}

/* phase transition */
void phasetransitionIHT(const std::string& filename, int maxn, int repitions,
int maxIterations, double tol1,double tol2 )
{
    std::ofstream demofile;
    demofile.open(filename);
    if (!demofile)
    {
        std::cout<<"unable to open file";
        return;
    }

    for(int m=1; m<=maxn; m++)
    {
        for(int k=1; k<=maxn; k++)
```

```
        {
            MMatrix A(m,maxn,0);
            MVector x0(maxn,0);
            MVector zeros(maxn,0);
            MVector x(maxn);
            double sum=0.0;
            for(int rep=0; rep<repitions; rep++)
            {
                A.initialise_normal();
                /*A.normalise_columns();*/
                x0=zeros;
                x.initialise_sparse_normal(k);
                sum+=successfulrecoveryIHT(A, x, x0, k, maxIterations, tol1, tol2);
            }
            demofile<<sum/repitions<<" ";
        }
    }
    demofile.close();
}

/* phase transition */
void phasetransitionNIHT(const std::string& filename, int maxn, int repitions,
int maxIterations, double tol1,double tol2 )
{
    std::ofstream demofile;
    demofile.open(filename);
    if (!demofile)
    {
        std::cout<<"unable to open file";
        return;
    }

    for(int m=1; m<=maxn; m++)
    {
        for(int k=1; k<=maxn; k++)
        {
            MMatrix A(m,maxn,0);
            MVector x0(maxn,0);
            MVector zeros(maxn,0);
            MVector x(maxn);
            double sum=0.0;
            for(int rep=0; rep<repitions; rep++)
            {
                A.initialise_normal();
                A.normalise_columns();
                x0=zeros;
                x.initialise_sparse_normal(k);
                sum+=successfulrecoveryNIHT(A, x, x0, k, maxIterations, tol1, tol2);
            }
            demofile<<sum/repitions<<" ";
        }
    }
    demofile.close();
}



int main()
{
    /*SLDS
    {
```

```
        MMatrix A(3,2,0.0);
        MVector x={0,0};
        A(0,0)=1; A(0,1)=2; A(1,0)=2; A(1,1)=1; A(2,0)=-1; A(2,1)=0;
        MVector b={10,-1,4};
        residualcontour("dat1.txt", A, b,-10, 10, -10, 10, 100, 100);
        b={10,-1,0};
        x={0,0};
        A(0,0)=1; A(0,1)=2; A(1,0)=2; A(1,1)=1; A(2,0)=-1; A(2,1)=0;
        residualcontour("dat2.txt", A, b,-10, 10, -10, 10, 100, 100);
        x={0,0};
        A(0,0)=1; A(0,1)=2; A(1,0)=2; A(1,1)=1; A(2,0)=1.8; A(2,1)=-2;
        residualcontour("dat3.txt", A, b,-10, 10, -10, 10, 100, 100);
        x={0,0};
        A(0,0)=1; A(0,1)=2; A(1,0)=2; A(1,1)=1; A(2,0)=-2; A(2,1)=-2;
        residualcontour("dat4.txt", A, b,-10, 10, -10, 10, 100, 100);
    }
    */

    /*Threshold
    {
        int k;
        MVector v={-9, 6, -7, 8, -5};

        k=3;


        MVector w(v.size(),0);
        auto itmin = std::min_element(v.begin(), v.end());
        int indexmin = std::distance(v.begin(), itmin);
        auto itmax = std::max_element(v.begin(), v.end());
        int indexmax = std::distance(v.begin(), itmax);

        for(int i=0;i<k;i++)
        {
            if(std::abs(*itmax)>std::abs(*itmin))
            {
                w[indexmax]=*itmax;
                v[indexmax]=0;
                itmax = std::max_element(v.begin(), v.end());
                indexmax = std::distance(v.begin(), itmax);
            }
            else
            {
                w[indexmin]=*itmin;
                v[indexmin]=0;
                itmin = std::min_element(v.begin(), v.end());
                indexmin = std::distance(v.begin(), itmin);
            }


        }

        v.Threshold(k);
        std::cout<<v<<std::endl;
    }
    */

    /*2x3 matrices
    {
        MMatrix A(2,3,0);
        A.initialise_normal();
```

```
        std::cout<<A<<std::endl;
        MVector x(3);            // creates a vector of size 10, initially all zeros
        x.initialise_sparse_normal(1); // fills 3 random entries with N(0,1) samples
        std::cout << x << "\n";
        MVector b=A*x;
        x={0,0,0};
        std::cout<<IHT(A,b,x,1,1000,1e-2)<<std::endl;
        std::cout<<x<<std::endl;
    }
    */

    /*projection illustration
    {
        MMatrix A(2,3,0);
        A(0,0)=1; A(0,1)=2;A(0,2)=3;A(1,0)=4;A(1,1)=5;A(1,2)=6;
        MVector x={0.5,0.2,1};
        MVector b=A*x;
        x={0.2,0,0};
        //std::cout<<IHT(A,b,x,1,1000,1)<<std::endl;
        //std::cout<<x<<std::endl;
        residual3dcontour("dat5.txt",A,b,0,1.5,0,1.5,0,1.5,100,100,100);
    }
    */

    /*projection illustration
    {
        MMatrix A(2,3,0);
        A(0,0)=0.6428; A(0,1)=1; A(0,2)=0.6428; A(1,0)=-0.7660; A(1,1)=0; A(1,2)=0.7660;
        MVector b={0.7428,0.7660};
        MVector x={0.6, 0.4, 0.3};
        std::cout<<IHT(A,b,x,1,100,0.15);
        residual3dcontour("dat7.txt",A,b,-0.5,1,-0.2,0.5,-0.2,1.2,100,100,100);
    }
    */

    /*successful recovery test
    {

        MMatrix A(2,3,0);
        MVector x0(3,0);
        MVector x(3);
        A.initialise_normal();
        std::cout<<A<<std::endl;
        x.initialise_sparse_normal(1);
        std::cout<<x<<std::endl;
        std::cout<<x0<<std::endl;
        std::cout<<successfulrecoveryIHT(A, x, x0, 1, 1000000, 0.1, 0.2)<<std::endl;
        std::cout<<x0<<std::endl;
    }
    */

    /*phase transition*/
    {
        phasetransitionIHT("phasetransition40_20IHT.txt", 40, 20, 100, 0.1, 0.2);
    }

    return 0;
}
```